D

Dumbhanengue ERP

08 — Flux métier

Ce document explique comment les cas d'usage se combinent dans les flux réels du système : **vente** (y compris son **annulation**), **crédit (« fiado »)**, **flux de trésorerie**, **stock** et **clôture de caisse du jour (Z-report)** — et les décisions métier derrière chacun.

Version 1.1.1+3 · 16 août 2026

Dumbhanengue ERP

Table des matières

1. Traitement de la vente (flux atomique) 3
   - Garanties 3
   - Snapshots de prix 3
   - Annulation de vente (annulation — `cancel_sale`) 4
   - Décisions métier de l'annulation 4
2. Le cycle du « crédit » (« fiado ») 5
   - Enregistrement (`AddDebtUseCase`) 5
   - Règlement (`SettleDebtUseCase`) 5
   - Visualisation 6
   - Impact sur le fonds de roulement 6
3. Flux de trésorerie (caisse réalisée) 6
   - Pourquoi `FinancialTransaction` est séparé de `Sale` ? 6
4. Contrôle du stock 7
   - 4.1 Total du produit (couche historique) 7
   - 4.2 Lots FIFO et péremption (W11) 7
   - 4.3 Ventes au poids (balances, W11) 7
5. Clôture de caisse du jour (Z-report) 7
   - Décisions métier 8

Documents connexes 9

1. Traitement de la vente (flux atomique)

> Deux chemins d'exécution, un contrat. Sur SQLite/en mémoire, le flux ci-dessous s'exécute dans un `IUnitOfWork` (une seule transaction). Sur Supabase, ProcessSaleUseCase délègue toute l'opération à la RPC `process_sale` (SECURITY DEFINER) — vente + articles + verrou/diminution de stock + entrée en caisse dans une transaction de base de données sur le serveur, avec idempotence sur l'id de la vente et les mêmes validations (stock, total en centimes). La description locale ci-dessous est le contrat métier ; la RPC le reflète (voir supabase/schema.sql §1.11). Il en va de même pour les achats (process_purchase), l'enregistrement de dettes (register_debt) et le règlement (settle_debt).

Le ProcessSaleUseCase intègre Ventes + Stock + Finances avec un ordre d'opérations délibéré, le tout dans un `IUnitOfWork` (une seule transaction — soit tout est écrit, soit rien) :

```
1. Validation de la vente
   rejette une vente sans articles
   rejette un total qui diverge de Σ(quantité × prix unitaire)
     comparaison en CENTIMES ((x * 100).round()) — évite les faux
       positifs de la double précision (ex. : 2 × 9.9 + 9.9)
2. Pré-validation du stock (AUCUNE mutation)
   pour chaque article : UpdateProductStockUseCase.ensureSufficientStock
     si manquant → InsufficientStockException → rollback, rien n'est écrit
3. Enregistrement de la vente (ISaleRepository.registerSale)
   SQLite : tables sales + sale_items dans la même transaction
4. Déduction du stock (par article)
   UpdateProductStockUseCase.decrease — revalide en interne
     (défense contre les appels directs) et persiste la VALEUR ABSOLUE
     (stock − quantité) via IProductRepository.updateStock
5. Entrée en caisse
   FinancialTransaction(income, catégorie "Vendas", statut paid)
```

Garanties

Garantie	Comment
Atomicité	IUnitOfWork → BEGIN IMMEDIATE / COMMIT / ROLLBACK (SQLite) ; no-op en mémoire (web)
Stock jamais négatif	UpdateProductStockUseCase est le seul port de mutation ; revalide d'abord ; le repository reçoit une valeur absolue ; CHECK (stock_quantity >= 0) comme dernière ligne de défense
Pas de vente « orpheline »	La pré-validation du stock se produit avant toute écriture
Pas de fenêtre TOCTOU	BEGIN IMMEDIATE acquiert le verrou d'écriture avant la pré-validation ; la file FIFO sérialise les transactions concurrentes dans le même processus
Pas de transaction imbriquée invalide	AppDatabase garde un compteur de profondeur réentrant (les appels imbriqués rejoignent la transaction externe)

Snapshots de prix

Chaque SaleItem conserve productName et unitPrice au moment de la vente (plus productId pour la traçabilité). Cela préserve :

- le prix historique (les futurs rapports montrent le montant facturé à l'époque) ;
- l'intégrité fiscale/comptable (le total ne change jamais rétroactivement) ;
- le vrai contexte de la transaction même après renommage/discontinuité des produits.

Annulation de vente (annulation — cancel_sale)

CancelSaleUseCase défait une vente enregistrée : il rend le stock aux lots exacts, annule la caisse et marque la vente comme annulée dans l'historique (elle reste visible avec un badge « Annulée », mais sort des rapports, des KPI et du résumé du jour). C'est une opération managériale (propriétaire/gérant) — les employés ordinaires ne peuvent pas annuler.

```
SaleDetailScreen (détail d'une vente ACTIVE + utilisateur propriétaire/gérant)
   confirmation (AlertDialog montre le montant de la vente)
 ▼
CancelSaleUseCase
  MODE SUPABASE (ISaleCancellationProcessor → RPC cancel_sale, UNE transaction) :
     délégation immédiate (pas de validation côté client) — le serveur
      valide l'existence + l'idempotence : garde is_manager_of_store
      (les employés NE PEUVENT PAS annuler — le serveur est l'autorité,
      même si un client modifié contourne l'UI)
     lit et VERROUILLE la vente FOR UPDATE (portée magasin) ; absente ou
      déjà annulée → no-op (ne lève jamais SaleNotFoundException)
     rend le stock aux lots EXACTS consommés (sale_lot_consumption) :
        par lot (FOR UPDATE) : current_quantity += rendu
          et le stock du produit += rendu — l'invariant
          `stock = Σ lots` ne se brise jamais
     annulation en caisse : SORTIE du montant de la vente, catégorie
      Ventes, statut paid, description « Annulation de vente — <moyen> » —
      la paire (entrée d'origine + annulation) ferme la caisse à ZÉRO
     marque la vente : status = 'cancelled', cancelled_at = now()
  MODE LOCAL (SQLite/en mémoire, dans un IUnitOfWork) :
     validation : getById → sinon SaleNotFoundException ; déjà
      annulée → retour (no-op idempotent)
     FifoStockUseCase.restoreLot pour chaque LotConsumption
     markCancelled(saleId, storeId)
     add FinancialTransaction(expense, « Annulation de vente — … », catégorie Ventes)
 ▼
Vente annulée → UI : badge « Annulée », total barré, reste dans l'historique
  SORT des rapports, des KPI et du résumé du jour (status != 'active' filtré)
  invalide salesListProvider, productsProvider, transactionsProvider,
   cashFlowProvider et dashboardKpisProvider (les écrans se rechargent)
```

Décisions métier de l'annulation

Décision	Pourquoi
Rend les lots EXACTS	La consommation de lots est enregistrée sur la vente (sale_lot_consumption) — l'annulation rend à chaque lot sa quantité d'origine, en préservant péremption et coût (pas de mélange de lots)
L'annulation ferme la caisse à zéro	L'entrée d'origine (Ventes, paid) + la sortie d'annulation (Ventes, paid) se compensent — la caisse du jour n'est pas gonflée par une vente annulée
La vente annulée reste dans l'historique	Piste d'audit : l'enregistrement reste visible avec le badge « Annulée » et le moment de l'annulation (cancelledAt)

Sort des rapports/KPI/résumé du jour	Toutes les agrégations filtrent status = 'active' (côté client et RPC) — une vente annulée ne compte jamais comme chiffre d'affaires
Idempotence	Annuler deux fois est un no-op — ne crée jamais d'annulation en double ni ne rend le stock deux fois (sur Supabase, une vente absente ou déjà annulée est aussi un no-op, sans exception)
L'annulation est managériale	Comme les achats et la clôture de caisse : is_manager_of_store sur le serveur ; canCancelSalesProvider dans l'app n'est qu'un guide UX
Le serveur est l'autorité sur SaaS	Le client n'envoie que sale_id + store_id ; la RPC valide la session, la garde et exécute l'annulation dans UNE transaction

> Vérification : sql_check.sh (17.x) exécute cancel_sale en tant qu'app_user et vérifie : lot/stock restaurés, cancelled_at rempli, seconde annulation sans doublons, la vente annulée hors des KPI/rapport (delta en centimes) et les employés rejetés quand ils annulent.

---

2. Le cycle du « crédit » (« fiado »)

Enregistrement (AddDebtUseCase)

1. Valide amount > 0 (InvalidDebtAmountException) ;
2. Enregistre la dette (IDebtRepository.add) — la source de vérité des à recevoir/à payer ;
3. Crée l'entrée en attente liée (debtId) :
    4. client (isCustomerDebt: true) → entrée en attente (à recevoir) ;
    5. fournisseur (isCustomerDebt: false) → sortie en attente (à payer).

> Si l'entrée en attente échoue après la dette, il lève PendingEntryRegistrationException — l'UI informe que le crédit a été enregistré et demande de vérifier Finances (jamais « réessayer », ce qui dupliquerait le crédit).

Règlement (SettleDebtUseCase)

```
1. Trouver la dette (sinon → DebtNotFoundException)
2. Idempotence : si déjà réglée, retour (ne duplique jamais l'entrée en caisse)
3. Marquer la dette comme payée (isPaid = true, paidAt = now)
4. Entrée en caisse :
   entrée en attente liée existe (debtId) ?
      en attente → marquer la MÊME transaction comme payée ET estampiller
            la date de réalisation (date = now — le flux de trésorerie
            reconnaît l'argent quand il ENTRE, en local comme en SaaS)
      déjà payée → rien (idempotent)
   n'existe pas → enregistrer une nouvelle entrée (catégorie « Fiado »)
```

> Sur Supabase, tout le règlement s'exécute comme la RPC `settle_debt` — une transaction, idempotente, portée par magasin. Depuis Aug 6, elle estampille aussi date = now() sur l'entrée en attente liée, en accord avec la sémantique locale SQLite markAsPaid.

Ordre délibéré : le règlement se produit avant l'entrée. Si quelque chose échoue au milieu, le pire cas est une dette réglée sans mouvement de caisse (livres sous-estimés) — jamais l'inverse, ce qui permettrait de facturer deux fois le client.

Visualisation

- GetPendingDebtsUseCase → à recevoir des clients pas encore réglés (onglet « En attente ») ;
- GetSettledDebtsUseCase → à recevoir des clients déjà réglés (onglet « Soldées », historique avec paidAt).

Impact sur le fonds de roulement

Les à recevoir sont du capital immobilisé (la marchandise est partie, mais l'argent n'est pas revenu). La structure permet : savoir combien dort chez les clients, planifier les achats sans dépendre du crédit, prioriser les encaissements par échéance et calculer le cycle financier.

---

3. Flux de trésorerie (caisse réalisée)

GetCashFlowUseCase calcule CashFlow(income, expense, balance) pour une période [start, end], en ne considérant que les transactions `paid` :

```
balance = Σ(entrées payées) − Σ(sorties payées)
```

- Une vente à crédit (entrée en attente) ne compte que lorsqu'elle est reçue ;
- Les à payer en attente ne sortent pas non plus de la caisse ;
- Quand une entrée en attente est reçue (MarkTransactionAsPaidUseCase / règlement de crédit), la date devient la date de réception — la réalisation en caisse se produit quand l'argent entre.

Pourquoi FinancialTransaction est séparé de Sale ?

	Sale	FinancialTransaction
Nature	Événement opérationnel (articles, stock, paiement)	Mouvement d'argent (sens, catégorie, montant)
Exemples	vente avec articles	vente, loyer, électricité, salaires, impôts, apports, crédit
Rôle	rapports opérationnels (marge, ticket moyen)	flux de trésorerie, entrées en attente, rapprochement

Les séparer évite les champs vides et les règles incohérentes (les dépenses n'ont ni article ni stock) et donne à chaque modèle une responsabilité unique.

4. Contrôle du stock

4.1 Total du produit (couche historique)

- `UpdateProductStockUseCase` est le port qui mute le total stock_quantity du produit :
  - ensureSufficientStock(productId, quantity) — pré-validation sans mutation ;
  - increase/decrease — revalident et persistent la valeur absolue ;
- `IProductRepository.updateStock` reçoit une valeur absolue (pas un delta) — deux couches de protection ;
- `CHECK (stock_quantity >= 0)` en SQLite est la dernière ligne de défense ;
- L'UI bloque l'ajout au panier au-delà du stock (soldOut) et signale le stock faible (< 5 unités) sur le tableau de bord et la liste.

4.2 Lots FIFO et péremption (W11)

- Chaque entrée en stock crée un `StockLot` (quantité initiale/actuelle en milli-unités, péremption facultative, coût d'acquisition en centimes). Les achats enregistrent une péremption facultative par article (purchase_items.expiry_date) — elle devient la péremption du lot ;
- La diminution au PDV consomme les lots en ordre FIFO — péremption la plus proche d'abord ; sans péremption, par date d'entrée (FifoStockUseCase localement, process_sale sur Supabase). Le surplus passe au lot suivant, et la somme des lots est pré-validée avant toute mutation (jamais de diminution partielle) ;
- Invariant : products.stock_quantity = Σ current_quantity des lots actifs — vérifié par sql_check.sh (test) et par la requête en lecture seule de sql_check_live.sh (live) ;
- Alertes de péremption sur le tableau de bord : rouges pour les lots périmés avec du stock, jaunes pour les lots expirant dans < 30 jours (expiryWarningDays), triées par urgence.

4.3 Ventes au poids (balances, W11)

- Les produits avec vendaPorFracao acceptent les quantités décimales au PDV (ex. 1.75 kg) — les quantités sont des milli-unités (1000 = 1 unité/kg), math entière exacte en Dart et SQL ;
- Les code-barres d'étiquette `20PPPPPWWWWW` sont interceptés au PDV : ScaleBarcode extrait le PLU (5 chiffres) et la valeur pesée, déduisant la fraction comme valeur ÷ prix (ne surfacture jamais). Interprété comme un code balance uniquement quand le produit existe et est vendaPorFracao — les codes EAN-12 commençant par 20 retombent sur la recherche normale ; les PLU sont normalisés (zéros de tête).

---

5. Clôture de caisse du jour (Z-report)

CloseCashRegisterUseCase enregistre la clôture du jour du magasin avec le comptage physique des espèces du tiroir. C'est une opération managériale (propriétaire/gérant) — les employés ordinaires ne peuvent pas clôturer. Le flux a deux phases : l'aperçu (lecture) et la confirmation (écriture).

> Aperçu côté serveur (H2) : le résumé du jour vient de la RPC get_cash_closing_summary (voir Architecture — Agrégations côté serveur) — l'app ne liste jamais l'historique des ventes pour additionner sur l'appareil. Cette section couvre la confirmation.

```
CashClosingSheet (Finances → Clôture de caisse)
  1. APERÇU — cashClosingSummaryProvider → get_cash_closing_summary (côté serveur)
     total du jour (tous les moyens) · attendu dans le tiroir (ESPÈCES uniquement)
       · répartition par moyen (JSONB) · clôture déjà enregistrée (si elle existe)
  2. L'utilisateur (propriétaire/gérant) saisit le COMPTAGE PHYSIQUE du tiroir
     différence EN DIRECT : juste / excédent / manque (vs. l'attendu)
  ▼
CloseCashRegisterUseCase
  valide countedCashCents >= 0 → sinon InvalidCashCountException
  MODE SUPABASE (ICashClosingProcessor → RPC close_cash_register, UNE transaction) :
     garde is_manager_of_store — les employés NE PEUVENT PAS clôturer (le
       serveur est l'autorité, même si un client modifié contourne l'UI)
     valide la session et le comptage non négatif SUR LE SERVEUR
     calcule SUR LE SERVEUR (le client n'envoie JAMAIS de valeurs calculées) :
        attendu = Σ des ventes ACTIVES EN ESPÈCES sur [p_day, p_day+1)
          · total = Σ des ventes actives (tous les moyens)
          · répartition = jsonb_object_agg par moyen (rapport Z)
     UPSERT dans cash_closings (UNIQUE store_id + closing_date) :
        re-clôturer le jour MET À JOUR le comptage physique, pas de doublons
     renvoie la clôture enregistrée (JSONB) → l'UI affiche le résultat
  MODE LOCAL (SQLite/en mémoire) :
     aggregateCashClosingDay (MÊMES critères que la RPC : actives
       uniquement, annulées exclues) → ICashClosingRepository.save (UPSERT par
       magasin + jour)
  ▼
Clôture persistée → l'UI affiche le résultat (sur SaaS, les valeurs affichées sont
  le RETOUR de la RPC — le serveur est l'autorité)
  invalide cashClosingSummaryProvider + cashClosingHistoryProvider
    (l'aperçu et l'historique sont re-récupérés avec la nouvelle clôture)
```

Décisions métier

Décision	Pourquoi
Attendu = espèces uniquement	Carte/M-Pesa/E-Mola/virement ne passent pas par le tiroir physique — l'attendu ne compare que ce qui devrait être en espèces
Comptage jamais négatif	Le propriétaire peut clôturer avec un tiroir vide, mais jamais avec une valeur négative (faute de frappe) — validé côté client et serveur
Les ventes annulées restent dehors	L'annulation (cancel_sale) a déjà ajusté la caisse — inclure les ventes annulées gonflerait la clôture (mêmes critères que les rapports et les KPI)
UPSERT par magasin + jour	Re-clôturer le jour (ex. : comptage mal saisi) met à jour le comptage et les snapshots, sans créer de ligne en double
La clôture est managériale	Comme les achats et l'annulation : is_manager_of_store sur le serveur — la matrice d'accès dans l'app (canCloseCashRegisterProvider) n'est qu'un guide UX

Le serveur calcule, le client n'envoie que le comptage	p_store_id, p_day (début du jour local en UTC) et p_counted_cash_cents — aucun total calculé ne quitte l'appareil
Critère unique aperçu/confirmation	aggregateCashClosingDay (local) reflète exactement ce que calcule la RPC — l'aperçu et la clôture ne divergent jamais

> Vérification : sql_check.sh (19.1–19.6) exécute close_cash_register/get_cash_closing_summary en tant qu'app_user et vérifie les valeurs (total, attendu, répartition, UPSERT sans doublons, employé rejeté à la clôture) ; sql_check_live.sh (35/35) valide la RPC en live ; les tests de contrat couvrent le payload et le mapping (cash_closing_supabase_data_source_test.dart) et le cas d'usage (close_cash_register_use_case_test.dart).

Documents connexes

- Domaine — cas d'usage et exceptions en détail.
- Couche de données — Unit of Work, schéma et transactions réentrantes.
- Présentation — comment les écrans consomment ces flux.